```
$ curl -v http://localhost:8080/index.php
< HTTP/1.1 200 OK
< Host: localhost:8080
< Date: Mon, 14 Jul 2025 17:47:03 GMT
< Connection: close
< X-Powered-By: none/of-your-business
< Content-type: text/html; charset=UTF-8
<
array(1) {
  ["X-Powered-By"]=>
  string(9) "PHP/8.4.6"
}
Thanks for visiting!
```

As you can see, we've changed the `X-Powered-By:` header to the new value, and we're no longer advertising the precise version of PHP powering our endpoint (and therefore no longer helping a malicious actor fine-tune an attack). Notice that we had to call `header()` before we dumped the stored output of `apache_response_headers()`. This could get quite tiresome and make for some tortuous logic if our code actually did real work…

Output Buffering

As demonstrated already, PHP will emit output immediately: any statement such as `print` or `echo`, or functions that can emit output such as `var_dump()` or `print_r()`, will trigger a status line and response headers when running under a web server SAPI. If you want the freedom to begin emitting output but make changes to HTTP status or headers, you will need to make use of PHP's output buffering mechanism. Let's recycle an earlier example of the problem. Remember that this code…

```
print "HTTP Status Code was " . http_response_code() . "\n";
print "But I don't like you so you are forbidden.\n";

http_response_code(403);
```

… caused PHP to emit a warning error message. We can get around this with the built-in output buffering functions. When output buffering is enabled (with the `ob_start()` function), bytes emitted by PHP as output will be stored in memory rather